src/flightCompV2/servos.cpp

```cpp
/*
 * Program to control ARC-S Servo's
 *
 * Name:      Thor Kopenkoskey
 * Created:   January 25, 2025
 * Edited:    November 17, 2025
 */

#include "servos.h"
#include "sensorfusion.h"
#include "PID.h"
extern craftState vehicle;

int initialOffset = 1500;
String inputString = "";

constexpr int SERVO_MIN_US = 500;
constexpr int SERVO_MAX_US = 2500;
constexpr int SERVO_CENTER_US = 1500;
constexpr int SERVO_AUTHORITY_US = 500; // ± authority from center


inline int clampServoUs(int us)
{
  return constrain(us, SERVO_MIN_US, SERVO_MAX_US);
}

AxisServo servos[] =
{
  {"+Y", Servo(), 3, initialOffset, 1},
  {"-X", Servo(), 4, initialOffset, 1},
  {"-Y", Servo(), 5, initialOffset, 1},
  {"+X", Servo(), 6, initialOffset, 1}
};

const int NUM_SERVOS = sizeof(servos) / sizeof(servos[0]);

void writeServoOffsetEEPROM(int servoIndex)
{
  int addr = servoIndex * 2;                  // 2 bytes per servo
  uint16_t val = servos[servoIndex].offset;  // current offset
  EEPROM.write(addr, val >> 8);              // high byte
  EEPROM.write(addr + 1, val & 0xFF);        // low byte
  EEPROM.commit();                             // save immediately
}

void loadOffsetsFromEEPROM()
```

```
{
  for (int i = 0; i < NUM_SERVOS; i++)
  {
    int addr = EEPROM_START + i * 2;
    uint16_t val = (EEPROM.read(addr) << 8) | EEPROM.read(addr + 1);
    if (val != 0xFFFF)
    { // ignore empty EEPROM
      servos[i].offset = val;
      servos[i].servo.writeMicroseconds(clampServoUs(val)); // center servo
      Serial.print("Loaded ");
      Serial.print(servos[i].name);
      Serial.print(" offset: ");
      Serial.println(val);
    }
  }
}

void processCommand(String cmd)
{
  cmd.trim();  // Remove whitespace
  if (cmd.length() == 0)
  {
    return;
  }

  int spaceIndex = cmd.indexOf(' ');
  if (spaceIndex == -1)
  {
    Serial.println("Invalid command. Use: ServoName Offset");
    return;
  }

  String servoName = cmd.substring(0, spaceIndex);
  String offsetStr = cmd.substring(spaceIndex + 1);
  int newOffset = offsetStr.toInt();

  bool found = false;
  for (int i = 0; i < NUM_SERVOS; i++)
  {
    if (servos[i].name == servoName)
    {
      servos[i].offset = newOffset;  // Update offset
      servos[i].servo.writeMicroseconds(clampServoUs(newOffset));  // Move servo
      Serial.print("Updated ");
      Serial.print(servoName);
      Serial.print(" to offset ");
      Serial.println(newOffset);
      writeServoOffsetEEPROM(i);                    // Save to EEPROM
      found = true;
```

```
      break;
    }
  }
}

// Max command length to avoid unbounded growth if no newline (e.g. binary stream or noise).
constexpr size_t MAX_SERIAL_INPUT_LEN = 64;

void readSerialAndUpdateServo()
{
  while (Serial.available() > 0)
  {
    char c = Serial.read();
    if (c == '\n')
    {
      processCommand(inputString);
      inputString = "";
    }
    else
    {
      if (inputString.length() < MAX_SERIAL_INPUT_LEN)
        inputString += c;
      // else drop byte to prevent heap exhaustion
    }
  }
}

void ControlServo(String name, float angle)
{
  for (int i = 0; i < NUM_SERVOS; i++)
  {
    if (servos[i].name == name)
    {
      int deltaUs = mapGyroToServo(angle, -90, 90);
      servos[i].writeDeltaUs(deltaUs);
      return;
    }
  }
}

float mapGyroToServo(float angle, float minAngle, float maxAngle)
{
  angle = constrain(angle, minAngle, maxAngle);

  return map(angle * 100,
             minAngle * 100, maxAngle * 100,
             -500, 500); // ±500 µs authority
}
```

```
// Control Loop

ServoMix servoMix;
ServoMix lastServoCommand;

float clamp(float value, float minValue, float maxValue)
{
    if (value < minValue) return minValue;
    if (value > maxValue) return maxValue;
    return value;
}

void flightControlUpdate(double dt)
{
    servoMix = {};              // clear accumulator

    pitchControlLoop(dt);
    yawControlLoop(dt);      // add pitch + yaw
    rollControlLoop(dt);          // add roll

    applyServoMix();            // send once
}

static PIDController pitchPid;

void initPitchController()
{
    // Output and integral limits are in "servo command degrees"
    const float out_min = -SERVO_LIMIT_DEG;
    const float out_max =  SERVO_LIMIT_DEG;

    const float i_min = -SERVO_LIMIT_DEG;
    const float i_max =  SERVO_LIMIT_DEG;

    pid_init(&pitchPid, KP_ROLL, KI_ROLL, KD_ROLL, out_min, out_max, i_min,
i_max);
}

void pitchControlLoop(double dt)
{
    int pitchSetpoint = 0;
    double pitchOrientation = vehicle.gyroAngle_deg.x;

    double pitchCmd = pid_update(&pitchPid, pitchSetpoint, pitchOrientation, dt);

    // Pitch → Y-axis servos
    servoMix.posY += pitchCmd;
    servoMix.negY += pitchCmd;
}
```

```
193
194 static PIDController yawPid;
195
196 void initYawController()
197 {
198     // Output and integral limits are in "servo command degrees"
199     const float out_min = -SERVO_LIMIT_DEG;
200     const float out_max =  SERVO_LIMIT_DEG;
201
202     const float i_min = -SERVO_LIMIT_DEG;
203     const float i_max =  SERVO_LIMIT_DEG;
204
205     pid_init(&yawPid, KP_ROLL, KI_ROLL, KD_ROLL, out_min, out_max, i_min, i_max);
206 }
207
208 void yawControlLoop(double dt)
209 {
210     int yawSetpoint = 0;
211     double yawOrientation = vehicle.gyroAngle_deg.y;
212
213     double yawCmd = pid_update(&yawPid, yawSetpoint, yawOrientation, dt);
214
215     // Yaw → X-axis servos
216     servoMix.posX += yawCmd;
217     servoMix.negX +=  yawCmd;
218 }
219
220 static PIDController rollPid;
221
222 void initRollController()
223 {
224     // Output and integral limits are in "servo command degrees"
225     const float out_min = -SERVO_LIMIT_DEG;
226     const float out_max =  SERVO_LIMIT_DEG;
227
228     const float i_min = -SERVO_LIMIT_DEG;
229     const float i_max =  SERVO_LIMIT_DEG;
230
231     pid_init(&rollPid, KP_ROLL, KI_ROLL, KD_ROLL, out_min, out_max, i_min, i_max);
232 }
233 void rollControlLoop(double dt)
234 {
235     int rollRateSetpoint = 0;
236     double rollRateMeasurement = vehicle.gyro_dps.z;
237
238     double rollCmd = pid_update(&rollPid, rollRateSetpoint, rollRateMeasurement,
    dt);
239
240
```

```
    servoMix.posX += rollCmd;
    servoMix.negX -= rollCmd;
}

void applyServoMix()
{
    // Clamp all outputs
    servoMix.posX = clamp(servoMix.posX, -SERVO_LIMIT_DEG, SERVO_LIMIT_DEG);
    servoMix.negX = clamp(servoMix.negX, -SERVO_LIMIT_DEG, SERVO_LIMIT_DEG);
    servoMix.posY = clamp(servoMix.posY, -SERVO_LIMIT_DEG, SERVO_LIMIT_DEG);
    servoMix.negY = clamp(servoMix.negY, -SERVO_LIMIT_DEG, SERVO_LIMIT_DEG);

    // Save commanded mix for telemetry before sending / resetting
    lastServoCommand = servoMix;

    // Send to servos
    ControlServo("+X", servoMix.posX);
    ControlServo("-X", servoMix.negX);
    ControlServo("+Y", servoMix.posY);
    ControlServo("-Y", servoMix.negY);

    // Reset for next cycle
    servoMix = {};
}

```